



BLM 3510

LIMITS OF ANN'S

AI Project 2026

23011918 LACHIN TALAEI

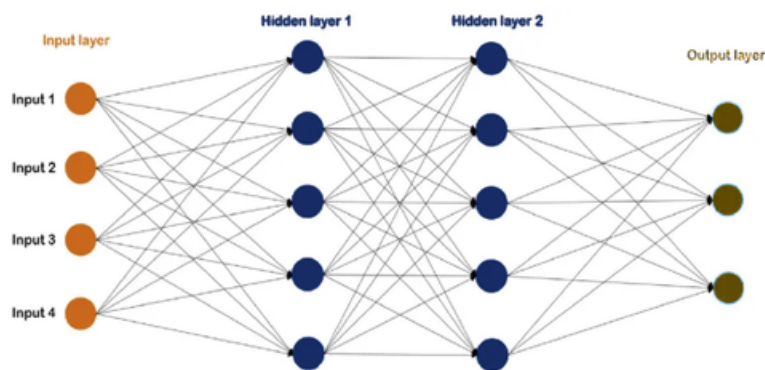
23011942 MARIAM SHERIF IBRAHIM

Vedio Link:

**[https://www.youtube.com/watch?
v=McNZWj_SUso&feature=youtu.be](https://www.youtube.com/watch?v=McNZWj_SUso&feature=youtu.be)**

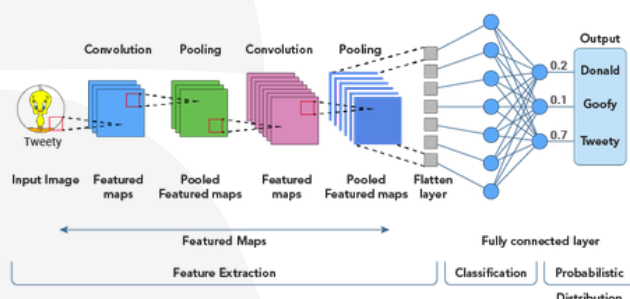
Introduction

One of the ways to improve AI is to try to imitate the amazingly designed human brain as much as possible. A neural network does just that! It is a machine learning model inspired by how the human brain works. It is made up of many layers artificial neurons (nodes) that process data and learn patterns. It is made up of 3 layers; the input layer, the hidden layer, and the output layer. Just like their names suggests, the input layer takes in data, the hidden layer does the calculations, and the output layer gives the final output.



There are more than one types of NN used in Artificial Intelligence. Each one has a unique feature that make them useful for different cases. ANN (Artificial Neural Network) and MLP (Multilayer Perceptron) are useful for structured data. CNN (Convolutional Neural Network) are best used for image and video analysis. RNN (Recurrent Neural Network) is made to work with sequential data.

Convolutional Neural Networks (CNNs) are special deep learning algorithms made for processing spatial data, like images and videos. Unlike traditional neural networks which turn images into a long line of pixel data, CNNs keep the picture's structure intact. They do this with mathematical filters that analyze tiny areas of the image at a time. This lets the network figure out complex visual patterns on its own – picking up basic stuff like edges and textures early on, then progressing to full shapes and objects in later layers. Since CNNs can recognize these visual features no matter where they show up in an image, they power lots of modern computer vision tech. Examples include facial recognition, analyzing medical images, and helping autonomous vehicles navigate their surroundings.



ANN

ARTIFICIAL
NEURAL
NETWORK

MLP

MULTILAYER
PERCEPTRON

CNN

CONVOLUTIONAL
NEURAL
NETWORK

RNN

RECURRENT
NEURAL
NETWORK

TRANS FORMER

ANN

ARTIFICIAL
NEURAL
NETWORK

MLP

MULTILAYER
PERCEPTRON

CNN

CONVOLUTIONAL
NEURAL
NETWORK

RNN

RECURRENT
NEURAL
NETWORK

TRANS FORMER

ABOUT THIS PROJECT

In this project, artificial neural networks (ANNs) that produce five different outputs in response to a 25x25 binary matrix input were trained and their results compared. The main goal of the study is not to "solve" a task, but to see which types of tasks ANNs can learn and which they fail at, and to interpret why these limitations arise. The project provided 5 different tasks, for each task we were required to create its data, upload it, run it through an ANN model, and do different hyperparameter optimization to find the what gave best results. The 5 tasks were:

- A) There are 5 points (in random positions) in the matrix. The model will output the Manhattan distance between the two closest points to each other.
- B) The same as A, only the two that are farthest from the closest place..
- C) There are N (between 1 and 10) points (in random positions) in the matrix with a variable number (there may be a different number in each entry). The model will output the number of points.
- D) Same as C, but output, whether the number of dots is odd or even
- E) Same as C but the number of exit and points is odd, the Manhattan distance of the point closest to the upper left corner to the upper left corner, and if it is even, the Manhattan distance of the point closest to the lower right corner to the lower right corner

All matrices are 25x25 and contain only the values {0,1}; "dots" are the cells that hold the value 1.

Code	Task	Type
A	Manhattan distance between the closest pair among 5 points	Regression
B	Manhattan distance between the farthest pair among 5 points	Regression
C	Count of N points ($1 \leq N \leq 10$)	Regression
D	Whether the number of points is odd or even	Binary Classification
E	If N is odd: Manhattan distance of closest point to top-left corner. If N is even: Manhattan distance of closest point to bottom-right corner	Regression

In this project we used the CNN model because because the input data is a 25x25 grid. This helps the model pick up on how pixels relate to each other. Also, since dots show up randomly, CNNs do are good at finding important patterns using shared filters – where the pattern is, doesn't matter.

DATA COLLECTION

How we created our data

Task A

Each sample contains 5 random points in a 25×25 binary matrix. We calculated all pairwise Manhattan distances and used the smallest distance as the label

Task B

Your paragraph textEach sample contains 5 random points in a 25×25 binary matrix. We calculated all pairwise Manhattan distances and used the largest distance as the label.

Task C

Each sample contains N random points, where N is between 1 and 10. The label is the number of points in the matrix.

Task D

Each sample is generated like Problem C, but the label shows whether the number of points is even or odd. We used 0 for even and 1 for odd.

Task E

Each sample contains N random points between 1 and 10. If N is odd, the label is the distance of the closest point to the upper-left corner. If N is even, the label is the distance of the closest point to the lower-right corner.

800 training + 200 test examples were generated for each problem. The output distribution was generated using constructive sampling to ensure it was as uniform as possible within each cluster; random placement and labeling would result in very frequent small distances and almost no large distances. Therefore, the target output value was pre-selected in A, B, and E, and a configuration was built to match it.

Model Architecture

Two CNN architectures were used in this project.

For Tasks A, B, C, and E (regression tasks), a two-layer CNN was used:

Component	Details
Conv Layer 1	1 → 16 filters, 3×3 kernel, padding=1
Conv Layer 2	16 → 32 filters, 3×3 kernel, padding=1
MaxPool	2×2 (applied after each conv layer)
Spatial size	25×25 → 12×12 → 6×6
FC Layer 1	1152 → 64 neurons
FC Layer 2	64 → 1 neuron (single value output)
Activation	ReLU
Loss Function	MSELoss
Optimizer	Adam (lr=0.001)

For Task D (classification), , a two-layer CNN was used:

Component	Details
Conv Layer 1	1 → 16 filters, 3×3 kernel, padding=1
MaxPool	2×2
Spatial size	25×25 → 12×12
FC Layer 1	2304 → 64 neurons
FC Layer 2	64 → 2 neurons (odd/even)
Activation	ReLU
Loss Function	CrossEntropyLoss
Optimizer	Adam (lr=0.001)

Hyperparameters Testing

For every task the testing accuracy for the following things were calculated:

- full_training_results: The results with all training data (32 batch, 0.001 LR) (default)
- half_training_results: The results with half the training data
- fourth_training_results: The results with fourth the training data
- 0.0001_training_results: The results with 0.0001 learning rate
- 0.01_training_results: The results with 0.01 learning rate
- 64_batch_training_results: The results with 64 batches
- 16_batch_training_results: The results with 16 batches

Task A:

Configuration	Result
Full Data (LR=0.001, Batch=32)	1.3980
Half Data	1.7393
Quarter Data	2.0818
LR = 0.0001	2.0663
LR = 0.01	1.2358
Batch Size = 64	1.4070
Batch Size = 16	1.1469

Task B:

Configuration	Result
Full Data (LR=0.001, Batch=32)	3.0109
Half Data	3.7125
Quarter Data	5.0124
LR = 0.0001	4.9699
LR = 0.01	3.0609
Batch Size = 64	4.6718
Batch Size = 16	2.9935

Task C:

Configuration	Result
Full Data (LR=0.001, Batch=32)	0.3155
Half Data	0.4999
Quarter Data	0.8329
LR = 0.0001	1.2884
LR = 0.01	0.2335
Batch Size = 64	0.4357
Batch Size = 16	0.2270

Task D:

Configuration	Result
Full Data (2 Layers)	0.5650
Full Data (1 Layer)	0.5600
Half Data	0.4950
Quarter Data	0.5850
LR = 0.0001	0.5700
LR = 0.01	0.5000
Batch Size = 64	0.5350
Batch Size = 16	0.5450

Task E:

Configuration	Result
Full Data (LR=0.001, Batch=32)	4.4436
Half Data	6.1004
Quarter Data	4.6610
LR = 0.0001	6.0366
LR = 0.01	4.4988
Batch Size = 64	4.3563
Batch Size = 16	4.4378

Note: Lower scores for A,B,C, and E is a better because they are loss scores
Higher percentage in D is better because it tests accuracy

Observations:

1. Data Sizes:

For all tasks the bigger the training dataset was the better the results

2. Learning rate:

For all tasks, 0.01 learning rate did the best. The lower the learning rate the worst it preformed

3. Batch Size:

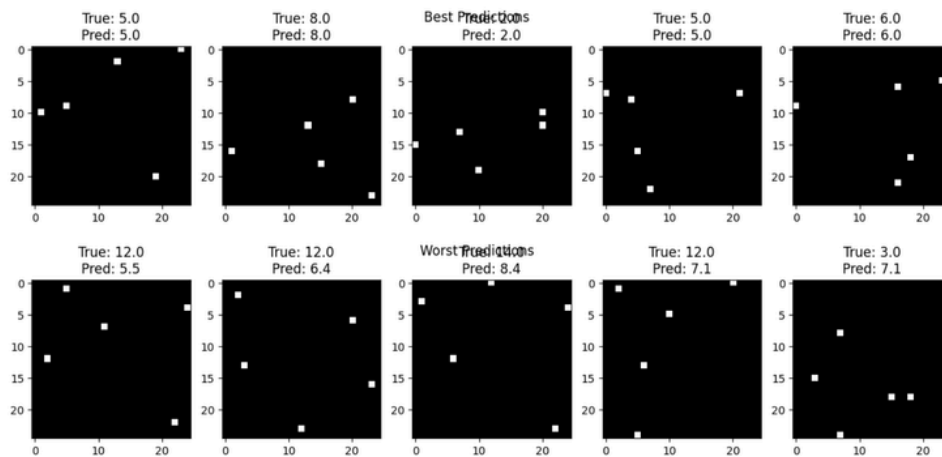
For all tasks, the batch size of 16 preformed the best

One key observation is the when the batch size was 16 it preformed best in almost all the tasks. This is likely because more frequent weight updates allow the model to learn more general patterns

Another observation is that D has about 0.5 for all testing cases. This is because the model is overfitted, the model is guessing 50/50 in the test. It is hard for the CNN to understand the even and odd pattern. All the other models worked and showed good results.

Observations based on matrixes

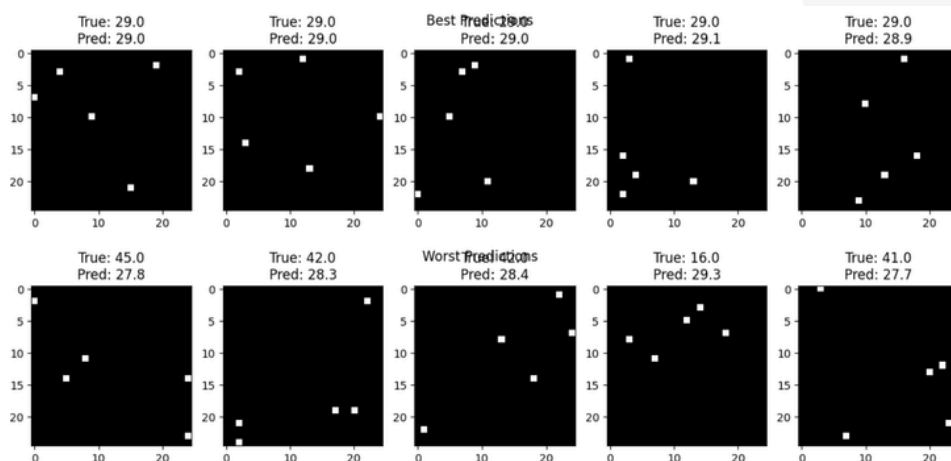
Task A:



For Task A, the CNN estimated the Manhattan distance between the two nearest points in a 25x25 binary matrix. The model did great with obviously easy visuals, figuring out rough spatial relations and giving accurate guesses for many tests. Still, it messed up badly when faced with confusing setups – like when there were lots of close pairs or when the nearest points blended in.

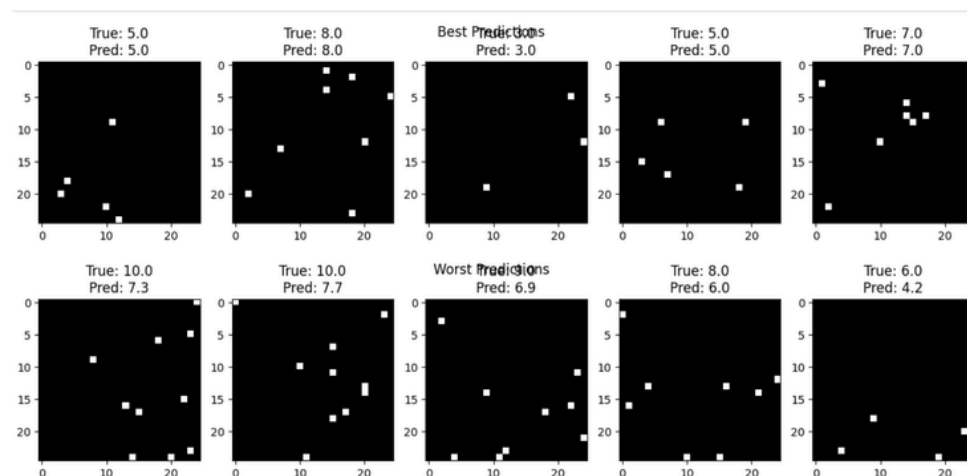
This points to the model using rough spatial guessing tricks instead of computing exact distances. It also showed a knack for averaging uncertain answers to medium values, not always aiming for the real distances. In sum, while the CNN could suss out broad spatial arrangements, exact geometry and relation stuff stumped it.

Task B:



Task B was way harder than Task A because it needed the model to find the closest pair of points first, figure out which one was farthest from those guys, and finally compute the Manhattan distance. Even so, the model did almost perfect on some examples but messed up on others, especially in complicated setups. Many times when it was wrong, its guesses were sorta okay but still off – leaning towards middling values, showing it wasn't sure and kinda regressing to the mean. It seemed like the CNN used rough spatial rules instead of figuring out the exact logic needed for the job. So, this shows a key limit of regular CNN designs – they ace spotting local patterns but stumble with precise multi-step reasoning tasks.

Task C:

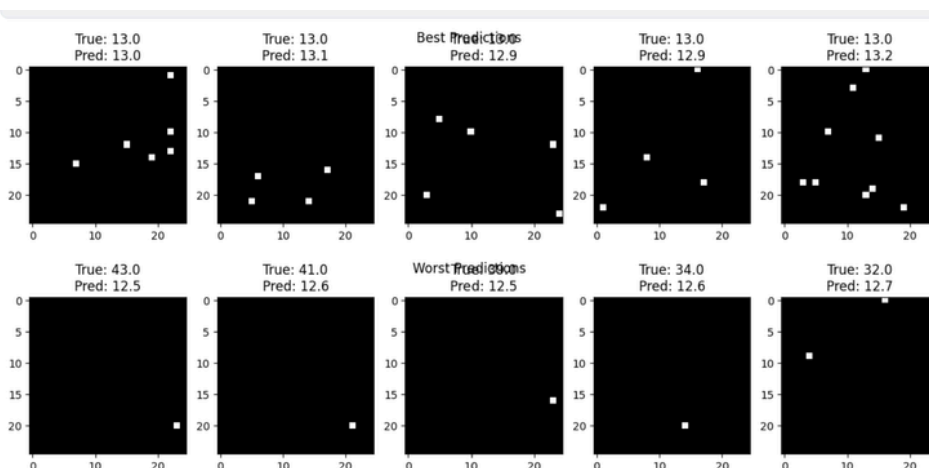


Task C performed way better than Tasks A and B since it simply involved counting points in a matrix, not complex relational or geometric thinking. The CNN learned strong visual links—how dense the activated pixels were—and matched those to the right output classes, making its predictions spot-on most of the time. Even where it messed up, the errors were minor, showing the model had grown pretty good at estimating quantities internally. Counting isn't like distance calculation; it doesn't need comparing things pairwise or following multi-step logic. So, it fits well with what CNNs excel at: recognizing patterns. This means standard CNNs ace rough counting and big picture feature tasks but fall short when precise step-by-step reasoning comes into play.

Task D:

In Task D, we saw severe overfitting; the model hit 98% training accuracy but only 56% on the test set – basically guessing randomly. Adjusting the hyperparameters didn't help; no matter what we did with the learning rate, batch size, or layer count, the results stayed the same. That consistency implies it's super tough for CNNs to handle the task, not some flaw in our design. Here's why: there's no real visual clue to tell 5 dots apart from 6. Thus, the network essentially memorized certain patterns from the training data rather than figuring out a way to count the dots generally. In essence, this shows a key limitation of CNNs: They excel at spotting pixel-level details, but they struggle with more abstract stuff, like the mathematical property of parity.

Task E:



The model excels when lots of dots are scattered across the grid, making nearly perfect predictions for distances around 13. It runs into trouble, though, with larger distances (32-43), often giving predictions of 12-13 instead. That's because those big distances happen only when there are hardly any dots – and they're all far from the corner. Since this is super rare in the training data, the model can't learn it. Instead, it sticks with what it sees the most – predicting around 12-13, basically the average distance it encountered. This shows a key limitation of CNNs: when faced with rare scenarios that result in extremes, they fail to adapt and just stick to safer, average guesses.